ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ И КОМПЈУТЕРСКО ИНЖЕНЕРСТВО

Еднодимензионални податочни структури

Алгоритми и податочни структури

Аудиториска вежба 5

Еднодимензионални податочни структури

- Со помош на основните податочни структури, можат да се креираат некои апстрактни податочни структури со специфична намена
  - стек(stack)
  - редица (queue)
  - приоритетна редица (priority queue)

Стек

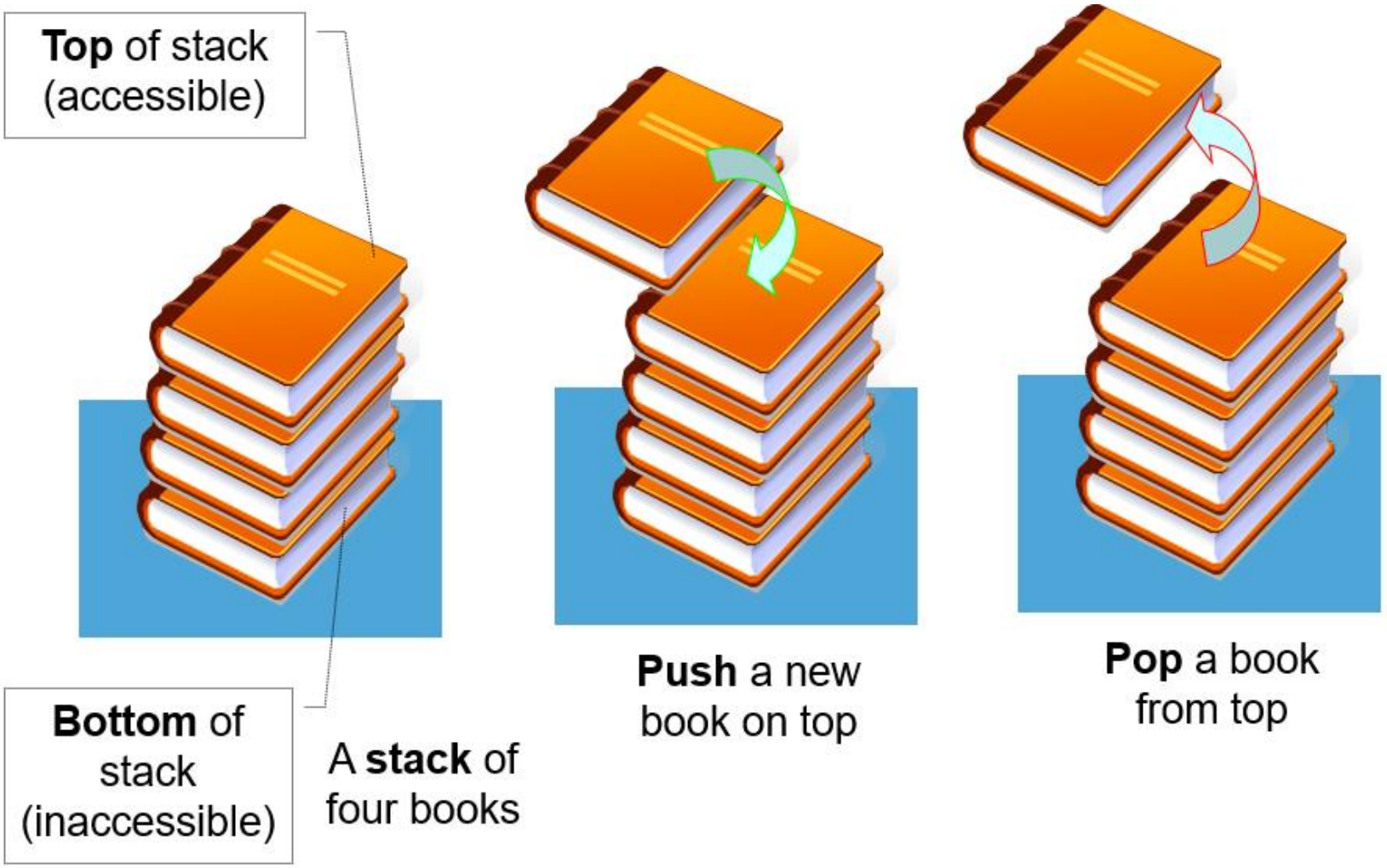
- **Стекот** работи на принципот LIFO (Last In – First Out).
- Во македонската литература се сретнува и со називот **Магацин**
- Операции:
  - за вметнување елемент на врвот на стекот – push
  - за бришење на елемент од врвот на стекот – pop

  - дополнително (во кодовите за Java) за да се прочита елементот на врвот од стекот, без притоа да се избрише - peek

Стек

- Како една од фундаменталните податочни структури стекот се користи во многу алгоритми и апликации
- Примери за употреба на стек
  - Backtracking алгоритми
  - Back/Forward функционалности во прелистувачи
  - Undo/Redo функционалности
  - Превртување на поврзана листа
  - Евалуација на аритметички изрази
  - Проверка на коректност на загради во израз
  - Парсери
  - Имплицитно во рекурзија
  - Повици на функции

Примена на стек



Имплементации на стек

- Постојат две генерални имплементации на стек:
  - Ограничен стек – кој се имплементира со користење на низа
  - Неограничен стек – кој се имплементира со користење на листа

Интерфејс за стек - Java

```
public interface Stack<E> {
    // Elementi na stekot se objekti od proizvolen tip.
    // Metodi za pristap:

    public boolean isEmpty();
    // Vrakja true ako i samo ako stekot e prazen.

    public E peek();
    // Go vrakja elementot na vrvot od stekot.

    // Metodi za transformacija:
    public void clear();
    // Go prazni stekot.

    public void push(E x);
    // Go dodava x na vrvot na stekot.

    public E pop();
    // Go otstranuva i vrakja elementot shto e na vrvot na stekot.
}
```

Се користи и во двете имплементации на стек

Стек со низа - Java

```
public class ArrayStack<E> implements Stack<E> {
        private E[] elems; //elems[0...depth-1] se negovite elementi.
        private int depth; //depth e dlabochinata na stekot

        public ArrayStack(int maxDepth) {
            // Konstrukcija na nov, prazen stek.
        }

        public boolean isEmpty() {
            // Vrakja true ako i samo ako stekot e prazen.
        }
        public E peek() {
            // Go vrakja elementot na vrvot od stekot.
        }

        public void clear() {
            // Go prazni stekot.
        }

        public void push(E x) {
            // Go dodava x na vrvot na stekot.
        }

        public int size() {
            // Ja vrakja dolzinata na stack-ot.
        }
        public E pop() {
            // Go otstranuva i vrakja elementot shto e na vrvot na stekot.
        }
    }
```

Стек со низа - Java

```
public boolean isEmpty() {
    // Vrakja true ako i samo ako stekot e prazen.
    return (depth == 0);
}

public E peek() {
    // Go vrakja elementot na vrvot od stekot.
    if (depth == 0)
        throw new NoSuchElementException();
    return elems[depth - 1];
}

public void clear() {
    // Go prazni stekot.
    for (int i = 0; i < depth; i++) elems[i] = null;
    depth = 0;
}
```

Стек со низа - Java

```
public void push(E x) {
    // Go dodava x na vrvot na stekot.
    elems[depth++] = x;
}

public int size() {
    // Ja vrakja dolzinata na stack-ot.
    return depth;
}

public E pop() {
    // Go otstranuva i vrakja elementot shto e na vrvot na stekot.
    if (depth == 0)
        throw new NoSuchElementException();
    E topmost = elems[--depth];
    elems[depth] = null;
    return topmost;
}
```

Стек со листа - Java

```
public class LinkedStack<E> implements Stack<E> {
        // top e link do prviot jazol ednostrano-povrzanata
        // lista koja sodrzi gi elementite na stekot .
        private SLLNode<E> top;
        int size;

        public LinkedStack() {    }

        public boolean isEmpty() {}

        public E peek() {}

        public void clear() {}

        public void push(E x) {}

        public int size() {}

        public E pop() {}
    }
```

Стек со листа - Java

```
public LinkedStack() {
    // Konstrukcija na nov, prazen stek.
    top = null;
    size = 0;
}

public boolean isEmpty() {
    // Vrakja true ako i samo ako stekot e prazen.
    return (top == null);
}

public E peek() {
    // Go vrakja elementot na vrvot od stekot.
    if (top == null)
        throw new NoSuchElementException();
    return top.element;
}

public void clear() {
    // Go prazni stekot.
    top = null;
    size = 0;
}
```

Стек со листа - Java

```
public E peek() {
    // Go vrakja elementot na vrvot od stekot.
    if (top == null)
        throw new NoSuchElementException();
    return top.element;
}

public void push(E x) {
    // Go dodava x na vrvot na stekot.
    top = new SLLNode<E>(x, top);
    size++;
}

public int size() {
    // Ja vrakja dolzinata na stekot.
    return size;
}

public E pop() {
    // Go otstranuva i vrakja elementot shto e na vrvot na stekot.
    if (top == null)
        throw new NoSuchElementException();
    E topElem = top.element;
    size--;
    top = top.succ;
    return topElem;
}
```

Класата `Stack` во Java

- Класата `Stack` претставува LIFO стек на објекти
- Ја наследува и проширува класата `Vector` со пет операции кои дозволуваат вектор да биде третиран како стек

  `public class` **`Stack<E>`** `extends` **`Vector<E>`**
- Опис на методите:
  - `empty()`
    - враќа: true ако и само ако стекот не содржи елементи; false во останатите случаи
  - `peek()`
    - враќа: објектот кој е најгоре на стекот (последниот елемент од Vector објектот), без притоа да го вади од структурата
  - `pop()`
    - враќа: објектот кој е најгоре на стекот (последниот елемент од Vector објектот и притоа го брише елементот)
  - `push(E item)`
    - враќа: item влезниот аргумент
  - `search(Object o)`
    - враќа: дистанцата до првото појавување на o од почетокот на стекот; -1 доколку објектот не е во стекот

Задача 1.

- Да се провери коректноста на заградите во еден израз.
- Еден израз има коректни загради ако:
  - За секоја лева заграда, подоцна следува соодветна десна заграда
  - За секоја десна заграда претходно постои лева заграда
  - Секој под-израз меѓу пар од две загради содржи коректен број на загради
- Примери на изрази со коректни и некоректни загради:

$s \times (s - a) \times (s - b) \times (s - c)$ — коректни

$(- b + \sqrt{[b^2 - 4ac]}) / 2a$ — коректни

$s \times (s - a) \times (s - b \times (s - c)$ — некоректни

$s \times (s - a) \times s - b) \times (s - c)$ — некоректни

$(- b + \sqrt{[b^2 - 4ac)}] / 2a$ — некоректни

Задача 1. - Java

```
public class Zagradi {
        public static boolean daliKorektni(String phrase) {
            // Test whether phrase is well-bracketed.
            ArrayStack<Character> bracketStack = new ArrayStack<Character>(100);
            for (int i = 0; i < phrase.length(); i++) {
                char cur = phrase.charAt(i);
                if (cur == '(' || cur == '[' || cur == '{')
                    bracketStack.push(cur);
                else if (cur == ')' || cur == ']' || cur == '}') {
                    if (bracketStack.isEmpty()) return false;
                    char left = bracketStack.pop();
                    if (!daliSoodvetni(left, cur)) return false;
                }
            }
            return (bracketStack.isEmpty());
        }
```

Задача 1. - Java

```
    public static boolean daliSoodvetni(char left, char right) {
        // Test whether left and right are matching brackets
        // (assuming that left is a left bracket and right is a right bracket).
        switch (left) {
            case '(':
                return (right == ')');
            case '[':
                return (right == ']');
            case '{':
                return (right == '}');
        }
        return false;
    }

    public static void main(String[] args) {
        String phrase = "s x (s - a) x (s - b) x (s - c)";
//        String phrase = "s x (s - a) x s - b) x (s - c)";
        System.out.println(phrase + " ima "
                + (daliZagraditeSePravilni(phrase) ? "korektni" : "nekorektni")
                + " zagradi.");
    }
}
```

Задача 2.

- ЗА ДОМА. Да се напише алгоритам кој ќе врши евалуација на израз во постфикс нотација.
- Пр. 5 9 + 2 * 6 5 * + изразот е во постфикс нотација

Редица

- **Редицата** работи на принципот FIFO (First In – First Out).
- Операции:
  - за вметнување на елемент на едниот крај на редицата т.е. на опашката на редицата – enqueue
  - за бришење на елемент на другиот крај на редицата, т.е. од главата (почетокот) на редицата– dequeue
  - дополнително (во кодовите за Java) за да се добие елементот на почетокот (главата) од редицата, без притоа да се избрише - peek

Редица - примена

- Кај мрежни рутери
- Плејлисти
- Кај мрежните печатачи, каде повеќе корисници испраќаат документи за печатење на делен печатач
- Кај хард дисковите, односно во нивните драјвери, каде повеќе процеси бараат пристап до податоците од дискот
- Во оперативните системи, каде повеќе процеси чекаат на ред да бидат опслужени од процесорот

Имплементации на редица

- Постојат две генерални имплементации на редица:
  - Ограничена редица – која се имплементира со користење на низа
  - Неограничена редица – кој се имплементира со користење на листа

Интерфејс за редица- Java

Се користи и во двете имплементации на редица

```
public interface Queue<E> {
        // Elementi na redicata se objekti od proizvolen tip.
        // Metodi za pristap:
        public boolean isEmpty();
        // Vrakja true ako i samo ako redicata e prazena.

        public int size();
        // Ja vrakja dolzinata na redicata.

        public E peek();
        // Go vrakja elementot na vrvot t.e. pocetokot od redicata.

        // Metodi za transformacija:

        public void clear();
        // Ja prazni redicata.

        public void enqueue(E x);
        // Go dodava x na kraj od redicata.

        public E dequeue();
        // Go otstranuva i vrakja pochetniot element na redicata.
    }
```

Редица со низа - Java

```
class ArrayQueue<E> {
        // Redicata e pretstavena na sledniot nacin:
        // length go sodrzi brojot na elementi.
        // Ako length > 0, togash elementite na redicata se zachuvani vo
    elems[front...rear-1]
        // Ako rear > front, togash vo  elems[front...maxlength-1] i elems[0...rear-1]
        E[] elems;
        int length, front, rear;

        // Konstruktor ...
        public ArrayQueue(int maxlength) {
            elems = (E[]) new Object[maxlength];
            clear();
        }

        public boolean isEmpty() {
            // Vrakja true ako i samo ako redicata e prazena.
            return (length == 0);
        }

        public int size() {
            // Ja vrakja dolzinata na redicata.
            return length;
        }
        public void clear() {
            // Ja prazni redicata.
            length = 0;
            front = rear = 0;  // arbitrary
        }
```

Редица со низа - Java

```
public E peek() {
        // Go vrakja elementot na vrvot t.e. pocetokot od redicata.
        if (length > 0)
            return elems[front];
        else
            throw new NoSuchElementException();
    }

    public void enqueue(E x) {
        // Go dodava x na kraj od redicata.
        if (length == elems.length)
            throw new NoSuchElementException();
        elems[rear++] = x;
        if (rear == elems.length) rear = 0;
        length++;
    }

    public E dequeue() {
        // Go otstranuva i vrakja pochetniot element na redicata.
        if (length > 0) {
            E frontmost = elems[front];
            elems[front++] = null;
            if (front == elems.length) front = 0;
            length--;
            return frontmost;
        } else
            throw new NoSuchElementException();
    }
}
```

Редица со листа - Java

```
public class LinkedQueue<E> implements Queue<E> {
       // Redicata e pretstavena na sledniot nacin:
        // length go sodrzi brojot na elementi.
        // Elementite se zachuvuvaat vo jazli dod SLL
        // front i rear se linkovi do prviot i posledniot jazel soodvetno.
        SLLNode<E> front, rear;
        int length;

        // Konstruktor ...
        public LinkedQueue () {
            clear();
        }

        public boolean isEmpty () {
               // Vrakja true ako i samo ako redicata e prazena.
            return (length == 0);
        }

        public int size () {
           // Ja vrakja dolzinata na redicata.
            return length;
        }
        public E peek () {
               // Go vrakja elementot na vrvot t.e. pocetokot od redicata.
            if (front == null)
                throw new NoSuchElementException();
            return front.element;
        }
```

Редица со листа - Java

```
public void clear () {
        // Ja prazni redicata.
         front = rear = null;
         length = 0;
    }
    public void enqueue (E x) {
        // Go dodava x na kraj od redicata.
         SLLNode<E> latest = new SLLNode<E>(x, null);
         if (rear != null) {
             rear.succ = latest;
             rear = latest;
         } else
             front = rear = latest;
         length++;
    }

    public E dequeue () {
        // Go otstranuva i vrakja pochetniot element na redicata.
         if (front != null) {
             E frontmost = front.element;
             front = front.succ;
             if (front == null)  rear = null;
             length--;
             return frontmost;
         } else
             throw new NoSuchElementException();
    }
}
```

Интерфејсот Queue во Java

- Нема готова класа за редица во Java, само интерфејс.
- Ова е `Queue` интерфејсот во Java:

```
public interface Queue<E> extends Collection<E>
{
        E element();
        boolean offer(E e);
        E peek();
        E poll();
        E remove();
}
```

Queue во Java

- Класи кои го имплементираат Queue интерфејсот се:
  - `AbstractCollection`
  - `LinkedList`
  - `PriorityQueue`
  - `LinkedBlockingQueue`
  - `BlockingQueue`
  - `ArrayBlockingQueue`
  - `LinkedBlockingQueue`
  - `PriorityBlockingQueue`

Задача 3.

- Студентската служба на факултет работи со студенти од 09:00 до 12:00. Ако знаеме за секој студент од редицата колку време е потребно за студентската служба да го опслужи, да се одреди кој студент ќе биде првиот студент кој нема да биде опслужен и колку студенти што чекале во редицата за тој ден не се опслужени.

Задача 3. - Java

```
class Student {
   String index;
   int minutes;

   Student(String index, int minutes) {
      this.index = index;
      this.minutes = minutes;
   }
}
public class StudentServiceQueue {
   static Student iterateStudentsQueue(ArrayQueue<Student> studentsQueue) {
      int minutes = 180;
      Student tempStudent = studentsQueue.peek();
      while (tempStudent != null) {
         if (minutes < tempStudent.minutes) {
            break;
         } else {
            minutes -= tempStudent.minutes;
            studentsQueue.dequeue();
         }
         tempStudent = studentsQueue.peek();
      }
      return tempStudent;
   }
```

Задача 3. - Java

```
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.println("Vnesi broj na studenti");
    int n = scanner.nextInt();
    ArrayQueue<Student> studentsQueue = new ArrayQueue<Student>(n);
    for (int i = 0; i < n; i++) {
        System.out.println("Vnesi index na studentot");
        String index = scanner.nextLine();
        System.out.println("Vnesi minuti potrebni za opsluzuvanje na studentot");
        int minutes = scanner.nextInt();
        Student student = new Student(index, minutes);
        studentsQueue.enqueue(student);
    }

    Student student = iterateStudentsQueue(studentsQueue);
    if (studentsQueue.isEmpty()) {
        System.out.println("Opsluzeni se site studenti");
    } else {
        System.out.println("Prviot student sto nema da e opsluzen e: " + student.index);
        System.out.println("Broj na studenti koi ne se opsluzeni za deneska: " + studentsQueue.length);
    }
}
```

Приоритетна редица

- Еднодимензионална линеарна секвенца од елементи, каде секој елемент има свој приоритет.
- Елементот со најголем приоритет секогаш прв се вади од листата
- Должина на приоритетна редица е бројот на елементи што ги содржи
  - Празната приоритетна редица има должина 0

Приоритетна редица

- Две можности при имплементација
  1. Подредена редица
  2. Неподредена редица
- И двете се независни од фундаменталниот податочен тип што ќе се користи (во однос на перформансите)
  - Со низа – ограничена приоритетна редица со предефиниран капацитет
  - Со поврзана листа – неограничена приоритетна редица